

Pricoins: A privacy-by-default cryptocurrency

Whitepaper, version 1 · 2026 · The Pricoins developers · github.com/pricoinsfoundation/pricoins

Abstract. Pricoins (ticker `PRIC`) is a Bitcoin Core fork in which every non-coinbase transaction is private at the consensus layer. Amounts are hidden by Pedersen commitments with rangeproofs; recipients are hidden by dual-key stealth addresses producing a fresh one-time output key per payment; senders are hidden by multi-layer CLSAG ring signatures with a fixed ring size of 16, paired with key images for double-spend prevention. The chain uses RandomX proof-of-work on a Monero-style 2048-block epoch seed rotation, a 150-second target block time, and an 840 000-block halving interval. Coinbase outputs remain transparent so the emission curve is auditable. Cross-chain atomic swaps with Bitcoin and Litecoin are supported natively via two parallel adaptor-signature protocols — Schnorr-adaptor / MuSig2 over P2TR for Taproot chains, and a discrete-log-bound HTLC for non-Taproot chains — both binding the foreign-leg secret to Pricoins's adaptor-CLSAG via the same elliptic-curve point T_G .

CONTENTS

- | | |
|--------------------------------|--------------------------|
| 1. Introduction | 11. BTC↔PRIC atomic swap |
| 2. Design goals | 12. LTC↔PRIC atomic swap |
| 3. Confidential amounts | 13. Discovery via Nostr |
| 4. Stealth addresses | 14. Wallet architecture |
| 5. CLSAG ring signatures | 15. Mining & pools |
| 6. Cooperative (joint) signing | 16. Threat model |
| 7. Adaptor-CLSAG | 17. Known limitations |
| 8. Transaction format (v4) | 18. Future work |
| 9. Consensus & PoW | 19. References |
| 10. Emission & supply | |

1. Introduction

Bitcoin's privacy properties degrade in two well-known ways. The transparent UTXO graph allows chain analysis to cluster addresses; and even when sophisticated users employ CoinJoin or similar mixing tools, the privacy gain is opt-in and benefits only those who can afford the friction. Honest users who pay no attention to coin selection routinely leak their transaction graph, balances, and counterparties to anyone running an indexer.

Several projects address this with privacy-by-default architectures — most notably Monero (CryptoNote/RingCT) and Zcash (zk-SNARKs). Pricoin sits between them: it inherits Bitcoin Core's codebase and operational maturity, replaces the transparent transaction format with a confidential-transactions bundle, and adopts the cryptographic primitives Monero has hardened over a decade — Pedersen commitments, stealth addresses, CLSAG ring signatures — but reuses Bitcoin's libsecp256k1 curve, RPC surface, P2P stack, and software- engineering practices. The result is a chain where every spend hides amount, recipient, and (probabilistically) sender, while the codebase remains close enough to Bitcoin Core that fixes and improvements upstream can be cherry-picked.

Pricoin additionally introduces native atomic swaps with Bitcoin and Litecoin via adaptor-signature protocols. The cross-chain binding is the same 33-byte compressed elliptic- curve point $T_G = t \cdot G$ on both sides; revealing t on either chain unlocks the counterparty leg on the other. No trusted bridges, no wrapped tokens, no escrows.

2. Design goals

- **Privacy is mandatory, not optional.** Spending a transparent input via the v1/v2/v3 paths returns `bad-non-pct-non-coinbase` at validation. There is no mode in which a Pricoin send leaks amount or recipient.
- **Auditable emission.** Coinbase outputs remain transparent (mining rewards are visible on chain). Total supply at any height is computable without scanning private outputs.
- **Bitcoin Core lineage.** Reuse the existing node software, P2P protocol, RPC surface, wallet architecture, and engineering process. Diverge only where privacy requires.
- **Native cross-chain interop.** Atomic swaps with BTC and LTC, no bridges. Same secret t binds both legs cryptographically.
- **Privacy-preserving discovery.** Order broadcast and peer-to-peer messaging happen over Nostr relays under the user's control. No centralized matching engine.

3. Confidential amounts

Pricoins replaces transparent `nValue` outputs with Pedersen commitments. A commitment to value `v` with blinding factor `r` is a curve point

$$C(v, r) = v \cdot H + r \cdot G$$

where `G` is the standard secp256k1 generator and `H` is a second generator with no known discrete-log relation to `G` (a "nothing-up-my-sleeve" point derived from `SHA256(G)` hashed to the curve). Commitments are homomorphic — addition of commitments equals commitment of summed plaintext values:

$$C(v_1, r_1) + C(v_2, r_2) = C(v_1+v_2, r_1+r_2)$$

so a transaction's balance equation $\sum \text{inputs} - \sum \text{outputs} - \text{fee} = 0$ can be verified under Pedersen commitments without revealing any individual amount, by checking that the sum of input commitments minus the sum of output commitments minus a transparent `fee · H` term commits to zero.

Rangeproofs

Pedersen commitments alone don't prevent a malicious sender from creating an output that "commits" to a negative value (which mod the group order looks identical to a positive value, but would inflate supply). Pricoins attaches a Borromean rangeproof to each output, proving that the committed value lies in $[0, 2^{64})$ without revealing it. Rangeproofs are ~1 280 bytes per output for the current Borromean construction (libsecp256k1-zkp's `rangeproof` module). Bulletproofs+ (BPP) adoption is on the roadmap and would cut this to ~100 bytes per output, but is gated on external review of the implementation in the secp256k1-zkp fork.

Encrypted amount + blinding

The recipient needs to recover both the value `v` and blinding `r` to spend the output. Pricoins encrypts these as part of the rangeproof, keyed by an ECDH shared secret derived from the sender's per-tx ephemeral keypair and the recipient's view key. Recovery is performed by attempting rangeproof rewind on each candidate output — a roughly constant-time operation that fails (cheaply) for outputs not addressed to the scanner.

4. Stealth addresses

A Pricoin recipient is identified by a stealth address — a pair of public keys (A, B) where $A = a \cdot G$ is the recipient's view key and $B = b \cdot G$ is their spend key. The address itself is reusable; revealing it does not link payments together.

For each output paying the address, the sender picks a fresh ephemeral scalar r , publishes $R = r \cdot G$ in the transaction, and computes a one-time output key

$$P = H_s(r \cdot A \parallel \text{index}) \cdot G + B$$

where H_s is a domain-separated hash to a scalar, $\cdot A$ is point multiplication of A by r , and index is the output's position in the transaction's vout array. The output's scriptPubKey commits to P ; on chain, every output to the recipient looks like a fresh, unrelated key.

Recipient scan: for each transaction, the recipient computes $S = a \cdot R$ using their view key, derives the same $H_s(S \parallel \text{index})$ shared secret, and checks whether the resulting one-time pubkey matches any output's scriptPubKey. On match, the spend key is

$$p = H_s(S \parallel \text{index}) + b \pmod{n}$$

Only the holder of b can derive p ; the view key alone is sufficient to recognize incoming payments but not to spend them.

The construction is the CryptoNote dual-key scheme as deployed by Monero. Pricoin's wallet uses a Bitcoin-style stealth address encoding (Base58Check, version byte `0x53`) with a 67-byte payload $(A \parallel B \parallel \text{checksum bits})$. Addresses begin with `H6`.

5. CLSAG ring signatures

A ring signature hides which member of an N-element pubkey set produced a signature, while still proving that one of them did. Pricoin uses CLSAG (Compact Linkable Spontaneous Anonymous Group signatures) — the same construction Monero adopted in 2020. Ring size is fixed at **16** in consensus; variable ring sizes leak fingerprints across transactions and are forbidden.

Single-layer CLSAG

Given a ring of $N=16$ public keys $\{P_0, \dots, P_{15}\}$ with the real signer at index π , single-layer CLSAG produces a signature $(c_0, s_0, \dots, s_{15}, KI)$ where $KI = x_\pi \cdot H_p(P_\pi)$ is the key image — a deterministic fingerprint of the real signer's private key, which is checked against the global key-image set for double-spend prevention.

Verification walks the ring:

$$\begin{aligned} L_i &= s_i \cdot G + c_i \cdot P_i \\ R_i &= s_i \cdot H_p(P_i) + c_i \cdot KI \\ c_{i+1} &= H_s(L_i \parallel R_i \parallel \text{msg} \parallel \text{ring}) \end{aligned}$$

and accepts iff c_0 equals the value derived after wrapping around the ring. Each non-signer's s_i is freshly random; the signer's s_π is computed so the equations close.

Multi-layer CLSAG (RingCT)

For confidential-amount inputs, the ring members are pairs (P_i, W_i) where $W_i = C_i - C_{\text{pseudo}}$ is the difference between the candidate's amount commitment and a sender-chosen pseudo-commitment. The signer's private witness is (x_π, z_π) where $z_\pi = \text{blind}_\pi - \text{blind}_{\text{pseudo}}$.

Pricoin's multi-layer CLSAG follows the standard Monero construction: tx-invariant key images ($KI = x_\pi \cdot H_p(P_\pi)$) and a separate commitment image $D = z_\pi \cdot H_p(P_\pi)$. Aggregation factors μ_P, μ_C bind the ring walk to (ring, KI, D) . Tx-invariant key images mean the same input signed under different pseudo-commitments produces the same KI, so double-spend prevention works across re-broadcasts. (An earlier draft used tx-dependent key images; this was a soundness gap and is no longer used.)

Decoy selection

Decoy quality determines the actual privacy gain — a ring where 15 of 16 members are obviously not the spender provides effectively no privacy. Pricoin uses Monero's gamma distribution over recent outputs as the decoy-selection function, drawing from outputs of compatible amount commitment and age. The full distribution is published with the wallet so it can be re-sampled by external auditors.

6. Cooperative (joint) signing

Atomic swaps require cooperative signing: two parties jointly produce a signature under a sum-aggregated pubkey $P_{\pi} = X_A + X_B$, where neither holds $x_{\pi} = x_A + x_B$ alone. Pricoin's joint CLSAG is a three-round protocol over a one-of-N ring containing the joint pubkey at the signer index:

1. **Round 1 — nonce gen.** Each party samples r_i and broadcasts L_i^{share} , R_i^{share} , KI_i^{share} , plus a commitment to a Schnorr DLEQ proof.
2. **Round 2 — combine.** Either party sums the shares, derives the standard CLSAG c_{π} , and broadcasts the per-party s_j for non-signer ring members and the partial s_{π}^{share} .
3. **Round 3 — assemble.** Each party computes their final s_{π}^{share} contribution, broadcasts; the aggregator sums into a complete CLSAG signature.

The protocol is implemented in `src/pricoi/joint_ringsig.{h,cpp}` for both single-layer and multi-layer rings. Self-tests at daemon startup exercise the full flow end-to-end against fresh randomness.

Rogue-key defense. Plain key aggregation is vulnerable to a rogue-key attack — a malicious party can choose X_B as a function of X_A to produce a joint pubkey for which they alone control the discrete log. Pricoin defends by requiring each party to attach a proof of possession (a Schnorr signature over a domain-separated tag with their pubkey) before accepting a joint pubkey. The PoP scheme is in `src/pricoi/joint_stealth.{h,cpp}`.

7. Adaptor-CLSAG

Adaptor signatures are the cryptographic primitive that makes atomic swaps work. Adaptor-CLSAG extends the standard CLSAG construction with an additional point $T_G = t \cdot G$ such that the signer can produce a pre-signature that verifies under the adaptor but is not a valid CLSAG until adapted with the secret t . Specifically:

- T_G — the public adaptor point on the standard generator G . Same byte representation as Bitcoin's compressed pubkey format; reused as-is on the cross-chain leg.
- $T_H = t \cdot H_p(P_{\pi})$ — the adaptor point on the per-output generator $H_p(P_{\pi})$. Required because CLSAG operates on two parallel generators.

- A DLEQ proof binding T_G to T_H via the same scalar t .

The pre-signature is a CLSAG-shaped tuple where the signer's L_π, R_π equations are satisfied modulo addition of T_G and T_H . $\text{Adapt}(\text{presig}, t) = \text{valid CLSAG}$. $\text{Extract}(\text{presig}, \text{signed_tx_sig}) = t$.

Cross-chain flow: the swap counterparty publishes the adapted signature on the PRIC chain to claim funds; the original signer extracts t from the on-chain CLSAG, then uses t to claim the foreign-chain leg whose script binds to T_G . Atomicity is cryptographic: anyone who can publish on one chain reveals t , which immediately unlocks the other.

The construction is in `src/pricoins/adaptor_ringsig.{h,cpp}` and its cooperative-signing wrapper in `src/pricoins/adaptor_joint_ringsig.{h,cpp}`. Both are exercised by the startup self-test gauntlet.

8. Transaction format (v4)

Pricoin defines transaction version `4` (`PRICOIN_CT_VERSION`) for confidential transactions. v1/v2/v3 spends — the entire Bitcoin transparent transaction family — are rejected for non-coinbase txs at consensus (`bad-non-pct-non-coinbase`). Only v4 is mineable on the PRIC chain, except for coinbases which remain v1.

A v4 transaction extends Bitcoin's `CTransaction` with a single `ct_bundle` structure:

```
CTBundle {
    input_commitments[] : Pedersen commitments to input amounts
                        (used in transparent-input mode for the
                        coinbase → CT on-ramp)
    ring_inputs[]       : RingCT inputs – N=16 candidate prevouts,
                        pseudo_commitment, multi-layer CLSAG sig
                        with key image
    outputs[]           : (commitment, rangeproof, scriptPubKey,
                        tx_pubkey, one_time_pubkey)
    transparent_fee     : explicit fee output, satoshis
}
```

The v4 sighash binds to the `ct_bundle` as a whole — any tampering with commitments, rangeproofs, or ring data invalidates every input signature.

UTXO model

Bitcoin's UTXO set is keyed by `(txid, vout)` with "spent" tracked by removal from the set. Pricoin's ring inputs reference 16 candidate prevouts, only one of which is the real input — so the standard "remove the spent UTXO" semantics don't apply. Pricoin replaces the UTXO set with two append-only structures:

- **Output set.** Every v4 output is added on confirmation and never removed, even after spending. Ring- decoy lookups need indefinite history.
- **Key-image set.** Every published key image is added on confirmation. Validation rejects any tx whose key image is already in the set (double-spend prevention).

Pruning is incompatible with this model — a spent output is still required as a potential decoy for future ring spends — and is rejected at startup (`-prune` returns `InitError`).

9. Consensus & PoW

Proof of work — RandomX

Pricoin uses RandomX (vendored from [tevador/RandomX](#)) as its proof-of-work function. RandomX is memory-hard and designed to make ASICs minimally advantageous, keeping mining accessible to commodity CPUs. The hash function input is the 80-byte serialized block header; the seed for the RandomX cache is rotated every `EPOCH_BLOCKS = 2048` blocks with a `EPOCH_LAG = 64` -block delay so miners can't precompute the next epoch too far ahead. The seed at height `h` is the block hash at `floor((h - lag) / 2048) * 2048`, with a fixed bootstrap seed for the first epoch.

RandomX outputs are 256-bit; the chain's block-identity hash is still `SHA-256d` (header) so block index keys remain Bitcoin-shape. Only `CheckProofOfWork` calls use the RandomX hash. Header validation across epoch boundaries handles the case where the seed-block isn't yet indexed during initial block download by deferring the PoW check to full-block validation, where the seed block has always been processed sequentially.

Block parameters

- **Target block time:** 150 seconds.
- **Difficulty retarget:** every 288 blocks (~12 hours), Bitcoin-style.
- **Halving interval:** 840 000 blocks (~4 years at target time).
- **Block weight:** raised from Bitcoin's 4 MWU to accommodate ~1 280-byte rangeproofs and ~600-byte ring sigs per output/input.
- **Initial reward:** 50 PRIC.
- **Total supply:** ~84 000 000 PRIC. Pricoin keeps Bitcoin's per-block reward (50 PRIC, halving every 840 000 blocks) but runs at 4× the block rate, so each era emits 4× as many coins as Bitcoin's corresponding era.

Genesis

Each network (mainnet, testnet, signet, regtest) has its own RandomX-mined genesis block, distinct `pszTimestamp`, and genesis hash (mainnet: `24e422e08dde...06a827f`, mined 2026-04-29). Genesis is bundled with the binary — no chain-history download is required to start syncing from height 0.

10. Emission & supply

Pricoin reuses Bitcoin's per-block-reward formula and halving interval but, because the block rate is 4× higher, the supply cap and per-era emission are also 4× larger. Block reward at height `h` is

$$\text{reward}(h) = 50 \text{ PRIC} \cdot 2^{-\lfloor h / 840000 \rfloor}$$

until the reward drops below one satoshi, after which it is zero. Total supply asymptotically approaches $\sim 84\,000\,000$ PRIC (era 0 alone emits $50 \times 840\,000 = 42\,000\,000$ PRIC, and subsequent eras halve as in Bitcoin). Coinbase outputs are **transparent**: amount visible, recipient P2WPKH, scriptPubKey on chain. This makes the cumulative emission auditable by inspection at any block height — a property the confidential-spend privacy model would otherwise lose.

The on-ramp from transparent coinbase $\rightarrow$ confidential is handled by `walletsendct` : transparent input(s) are consumed and the output is a v4 CT bundle. From that point on, the value lives in confidential outputs and is spent through RingCT. There is no path from a confidential output back to a transparent one short of mining.

11. BTC↔PRIC atomic swap

Pricoin's BTC swap protocol uses MuSig2 + Schnorr-adaptor on the BTC leg paired with cooperative adaptor-CLSAG on the PRIC leg. The cross-chain binding is the same point T_G on both sides. Both legs use libsecp256k1's BIP340 / BIP341 paths so the BTC outputs are indistinguishable from any other 2-of-2 P2TR.

Roles

We label the two parties by their role in the swap: **PRIC seller** sells PRIC, buys BTC; locks PRIC into a joint stealth output, eventually claims BTC by adapting a pre-signed BTC claim transaction. **PRIC buyer** picks the secret t , locks BTC into a 2-of-2 P2TR output, eventually claims PRIC by adapting a pre-signed PRIC claim CLSAG.

Setup

1. Parties exchange their swap-identity x-only pubkeys via Nostr DM. The PRIC seller chooses an ephemeral r ; together they compute the joint stealth address $(A_A + A_B, B_A + B_B)$ and the corresponding one-time pubkey P_π for the PRIC funding output.
2. The PRIC buyer picks t uniformly at random, computes $T_G = t \cdot G$, $T_H = t \cdot H_p(P_\pi)$, and a DLEQ proof binding the two. They send (T_G, T_H, π_{DLEQ}) to the PRIC seller.
3. Both parties agree on refund timelocks (t_{BTC}, t_{PRIC}) with $t_{BTC} > t_{PRIC} + \Delta_{min}$, where Δ_{min} is large enough for the seller to extract t from a confirmed PRIC claim and broadcast a BTC claim before the BTC refund becomes spendable.

Funding

The PRIC buyer funds the BTC leg as a 2-of-2 P2TR output whose aggregate x-only pubkey is $\text{MuSig2.Aggregate}(X_A, X_B)$. The PRIC seller funds the PRIC leg by sending the agreed amount to the joint stealth address using `walletsendct` with the pre-agreed ephemeral r , producing the joint output at P_π .

Pre-signing

Before either funding tx broadcasts, both parties cooperatively pre-sign four transactions:

- **BTC claim:** a Schnorr-adaptor pre-signature over T_G . Verifies under the aggregate x-only pubkey via standard BIP340; not a valid Schnorr sig until adapted with t .
- **BTC refund:** a plain MuSig2 signature timelocked to t_{BTC} . Broadcastable if the swap aborts.

- **PRIC claim:** a cooperative adaptor-CLSAG pre-signature over T_G / T_H .
- **PRIC refund:** a plain cooperative CLSAG timelocked to t_{PRIC} .

Claim

The PRIC buyer claims first. They adapt the PRIC claim pre-sig with their held t , producing a valid CLSAG, and broadcast the PRIC claim transaction. The published CLSAG signature reveals t to anyone who holds the matching pre-signature — specifically, the PRIC seller. The seller extracts t via `adaptor_ringsig::Extract(ring, presig, sig)`, adapts the BTC claim pre-sig with t to produce a valid BIP340 signature, and broadcasts the BTC claim transaction.

Atomicity is cryptographic. The PRIC buyer cannot claim PRIC without revealing t on chain. The PRIC seller's BTC claim is signed under the same t ; if the buyer never claims PRIC, the seller waits for t_{PRIC} and broadcasts the PRIC refund. If the seller never broadcasts the BTC claim, the buyer waits for t_{BTC} and broadcasts the BTC refund. Neither party can take both legs.

12. LTC↔PRIC atomic swap

Litecoin does not have Taproot, so the BTC swap's MuSig2 + Schnorr-adaptor flow over P2TR cannot be used directly. A naive substitute — a SHA-256 hashlock HTLC bound to $H_t = \text{SHA256}(t)$ — has a subtle atomicity flaw: there is no way to prove $H_t = \text{SHA256}(t)$ where $t = \text{dlog}(T_G)$ in zero knowledge without a SNARK over SHA-256. A malicious PRIC buyer can publish a fabricated $H_{t'}$, claim PRIC normally (revealing the real t), and leave the PRIC seller unable to satisfy the LTC HTLC.

Pricoins's LTC protocol uses a discrete-log-bound HTLC instead. The LTC HTLC redeem script is:

```
OP_IF
  <PRIC_seller_pubkey> OP_CHECKSIGVERIFY
  <T_G_compressed> OP_CHECKSIG
OP_ELSE
  <timeout> OP_CHECKLOCKTIMEVERIFY OP_DROP
  <PRIC_buyer_pubkey> OP_CHECKSIG
OP_ENDIF
```

The claim path requires two ECDSA signatures: one under the seller's pubkey (proving they're the seller) and one under T_G itself. Anyone who knows $t = \text{dlog}(T_G)$ can construct the second signature (treating t as the privkey for T_G). Knowledge of t is enforced by the script verifier, not by an off-chain hash relation; no SNARK or trust assumption is required.

The cross-chain binding is identical to BTC: the same T_G appears in the LTC redeem script and as the adaptor point for the PRIC adaptor-CLSAG. When the PRIC buyer claims PRIC, t is revealed on the PRIC chain and the seller uses it to build the LTC claim witness $[T_G\text{-sig}, \text{seller_sig}, 0x01, \text{redeem_script}]$.

Privacy tradeoff. The DL-HTLC's redeem script has a recognizable on-chain shape (two `OP_CHECKSIG` on the IF branch, CLTV on the ELSE), distinguishable from a generic 2-of-2 multisig. A future hardening pass could replace this with a 2-of-2 P2WSH funded under a cooperative ECDSA + ECDSA adaptor protocol, achieving the same indistinguishability as the BTC P2TR leg, but at substantial cryptographic-engineering cost (Paillier or class-group encryption, multiple zero-knowledge proofs). The DL-HTLC is the v1 design; cooperative-ECDSA is on the roadmap.

13. Discovery via Nostr

Pricoins's orderbook and peer-to-peer messaging layer rides on Nostr (Notes and Other Stuff Transmitted by Relays), a lightweight pub-sub protocol over WebSockets. There is no centralized matching engine. Any user can run a relay; the Pricoins foundation operates a four-node mesh (`relay1..4.pricoin.io`) cross-streamed via strfry so an event ingested anywhere replicates to all four within hundreds of milliseconds.

Order events (kind 30030)

Offers are published as Nostr parameterized replaceable events (NIP-33) at kind `30030` :

```
{
  "kind":      30030,
  "pubkey":    <maker's swap-identity x-only pubkey, hex>,
  "tags": [
    ["d",      "<order_id>"],
    ["expiration", "<unix_sec>"],
    ["c",      "btc" | "ltc"],
    ["s",      "buy" | "sell"]
  ],
  "content": "pricooffer:v1/<base64-payload>",
  "id":      sha256(canonical [0, pubkey, ts, kind, tags, content]),
  "sig":      BIP340 sig over id by maker's swap-identity priv
}
```

The base64 payload encodes the offer's full parameters: side, amounts, foreign-chain identifier, expiry, maker pubkey, and an internal ECDSA signature. Both the Nostr event signature and the URI's internal signature are validated; a mismatch is rejected.

Direct messages (kind 4 / NIP-04)

Peer-to-peer coordination during a swap (joint stealth handshake, match notifications, swap-start params, on-chain sig + ring after a claim) flows through NIP-04 encrypted DMs. The shared key is the X-coordinate of $ECDH(my_swap_priv, peer_xonly_pub \rightarrow even-y)$, the same construction Bitcoin's adaptor-MuSig2 uses for binding. Content is AES-256-CBC + random IV.

Pricoins layers an application-level ACK on top of NIP-04: each structured DM carries a `dm_id` field; the receiver auto-emits `{"type":"pricoins:ack/v1", "ack_for":<dm_id>}` on receipt. Senders can surface "Awaiting receipt" / "Counterparty received" status to the user. Without ACKs, "delivered to relays" is the only delivery signal, which says nothing about whether the counterparty's wallet processed the message.

Pricoins relay mesh

The four foundation-operated relays run strfry behind nginx with Let's Encrypt certs. Each relay's writepolicy filter accepts only kinds `4, 5, 30030`, keeping the mesh focused on Pricoins traffic and avoiding general Nostr noise. Cross-streaming uses `strfry stream --dir both` in a 4-node, 6-edge full mesh: each relay runs three stream daemons (one per peer); any single-node failure leaves the surviving three fully synced.

Users are not required to use the foundation relays. The relay list is a wallet setting; any Nostr relay that accepts kind `30030` works. Self-hosting is encouraged for stronger privacy.

14. Wallet architecture

A Pricoin wallet is built on Bitcoin Core's wallet code with a stealth-identity layer added on top. The user's persistent secret is a stealth seed: a 32-byte random value from which the view key, spend key, and per-purpose sub-identities are deterministically derived via HMAC chains.

Key derivation

The stealth seed produces:

- **View key** `a` and **spend key** `b` — the dual-key stealth address pair. Stored in `wallets/<name>/pricoind_stealth.dat`.
- **Swap-identity key** — used as the maker pubkey in orderbook events, the Nostr identity, and the cooperative-signing key for atomic swaps. Derived as `HMAC-SHA256(spend_seed, "pricoind/swap/identity-v1")`.
- **Holding-wallet keys** — one per foreign chain. Derived as `HMAC-SHA256(spend_seed, "pricoind/holding/<chain>-v1")`. Used to sign foreign-chain funding/sweep transactions.

Restoring from the stealth seed alone recovers all derived keys, including the foreign-chain holding wallets. Users retain full custody of their BTC and LTC at all times — the holding wallets are simplified by design (no address book, no fee estimation for arbitrary spends, no multi-recipient sends), but the keys are theirs.

Holding wallets

Cross-chain swaps require the user to deposit BTC or LTC at a per-chain receive address before the swap can fund. The holding wallet supports:

- **Receive** — single deterministic P2TR (BTC) or P2WPKH (LTC) address.
- **Balance query** — via the configured Esplora-style HTTP backend.
- **Sweep** — send all UTXOs to an external destination, fee suggested from the backend's `/fee-estimates`.
- **Atomic-swap funding** — invoked by the swap protocol; not user-callable directly.

Default backends are `https://blockstream.info/api` (BTC) and `https://litecoinspace.org/api` (LTC). Users concerned about address-clustering by these third-party services can replace either with a self-hosted Esplora instance via the `-btcwatchurl` / `-ltcwatchurl` daemon flags.

15. Mining & pools

Solo mining via `pricoind`'s built-in `generatetoaddress` RPC works for regtest / development. For mainnet, the foundation operates a PPLNS pool reachable at `stratum.pricoin.io:3333` (Stratum V1 over plain TCP). Cloudflare cannot proxy raw Stratum, so the pool runs behind DNS-only routing.

Pool architecture

The pool is implemented in Python 3 with a C extension (`rxshare`) wrapping `libRandomX` for fast share verification. Components: `pool.rpc` (cookie + rpcauth-aware JSON-RPC client), `pool.db` (SQLite WAL schema for workers/shares/blocks/payouts), `pool.pplns` (sliding-window split with largest-remainder rounding), `pool.vardiff` (per-connection difficulty retarget), `pool.jobs` (getblocktemplate → 80-byte blob with real coinbase + merkle and a 4-deep recent template ring), `pool.stratum` (asyncio TCP), `pool.payouts` (batched `walletsendct_multi`), `pool.web` (Flask /api stats).

Miner support

Pricoins uses Bitcoin's 80-byte header layout, which differs from Monero's 39-byte hash blob. The reference miner is a fork of [xmrig](#) at v6.26.0 with ~150 lines of patches: a new `rx/pric` algorithm enum, a 76-byte nonce offset (vs Monero's 39), a Pricoins coin entry, and OpenCL kernel updates. The binary is `pricoins-miner` ; build with `cmake -DWITH_HWLOC=OFF -DWITH_OPENCL=OFF -DWITH_CUDA=OFF` for the simplest CPU-only configuration.

16. Threat model

What's protected

- **Amount privacy.** An observer with the full chain history cannot recover the value of any non-coinbase output beyond proving it lies in $[0, 2^{64})$. Total emission is auditable from coinbase outputs only.
- **Recipient privacy.** Outputs paying the same stealth address are unlinkable on chain. The recipient's identity is concealed even from the sender after the transaction confirms.
- **Sender privacy (probabilistic).** The real signer of a CLSAG is hidden among 16 ring members. Effective privacy depends on decoy-selection quality — Pricoin uses Monero's gamma distribution.
- **Cross-chain swap atomicity.** Either both legs settle, or both refund. No party can take both sides; the cryptographic binding via T_g makes this a property of the protocol, not of trusted intermediaries.

What's not protected

- **Network-level metadata.** Pricoin nodes broadcast transactions over a P2P gossip protocol the same as Bitcoin. An adversary with broad network observation can link transactions to source IPs unless the user runs over Tor / I2P. `-listenonion` is supported.
- **Wallet metadata.** The wallet's view key scans every confirmed v4 output to find incoming payments. Anyone with the view key can read incoming amounts (but not spend). The wallet stores per-tx history including recovered amounts. A device compromise reveals everything the wallet has scanned.
- **Decoy-selection failures.** If the gamma distribution becomes a poor model of real spending behavior (e.g., due to a sudden shift in the age distribution of outputs), real signers can become statistically identifiable. Periodic distribution review is on the roadmap.
- **Foreign-chain privacy on swaps.** The PRIC leg of an atomic swap is fully confidential. The BTC leg is a 2-of-2 P2TR — indistinguishable from any other key-path Taproot spend. The LTC leg's DL-HTLC has a recognizable script shape on chain; mitigations are documented above ([\\$ LTC swap](#)).
- **Order-discovery metadata.** Nostr relays see the maker pubkey on every published order. The pubkey is per-wallet, not per-order, so a malicious relay can link all orders by the same maker. Self-hosting your own relay addresses this; subscribing only to your own relay does not (you'd still see no orders from anyone else).

17. Known limitations

Pricoins is experimental research code. Several components have not received external cryptographic review:

- **Adaptor-CLSAG.** The adaptor-signature construction over CLSAG is novel; it follows known adaptor-Schnorr techniques but the multi-layer extension has not been peer-reviewed. Suitable for testnet research only at present.
- **Cooperative joint CLSAG.** Three-round protocol with PoP rogue-key defense. The joint case amplifies the risk surface relative to single-party CLSAG. Implementation passes an internal end-to-end self-test gauntlet but lacks external audit.
- **Borromean rangeproofs.** ~1 280 bytes per output is large. Bulletproofs+ in libsecp256k1-zkp would cut this by an order of magnitude but the BPP module upstream is marked experimental.
- **LTC swap script fingerprint.** See § [LTC swap](#) — the DL-HTLC has a distinctive on-chain shape. Cooperative-ECDSA + ECDSA- adaptor would close this gap; deferred to v2.
- **Mainnet at low difficulty.** The chain is new; difficulty has not yet stabilized at a level that would deter a well-resourced adversary from briefly attaining majority hashrate. Confirmations should be deeper than Bitcoin's 6-block convention until mainnet hashrate is established.

18. Future work

- **Bulletproofs+ rangeproofs** — gated on upstream stabilization in libsecp256k1-zkp.
- **Cooperative ECDSA + ECDSA-adaptor** for the LTC leg, achieving the same indistinguishability as the BTC P2TR funding output.
- **DOGE / BCH atomic swaps.** The DL-HTLC primitive is chain-neutral; adding a new non-Taproot chain is mostly Esplora plumbing and a network-params entry.
- **Decoy-distribution review.** Monitor on-chain spending behavior; if the gamma distribution drifts away from observed reality, retune.
- **Mobile wallet.** A lightweight client for the orderbook + DM layer that can participate in matching without a full node.
- **External cryptographic review.** Adaptor- CLSAG and the joint signing protocols deserve an independent academic review before mainnet sees non-trivial value.

19. References

1. Goodell, Noether, RandomX: RandomX. tevador, 2019. github.com/tevador/RandomX
2. Maxwell, G. Confidential Transactions. 2015. elementsproject.org
3. van Saberhagen, N. CryptoNote v 2.0. 2013. Original stealth-address construction.
4. Goodell, Noether, RandomRun. Concise Linkable Ring Signatures and Forgery Against Adversarial Keys (CLSAG). 2019.
5. Bünz, B. et al. Bulletproofs: Short Proofs for Confidential Transactions and More. 2018.
6. Nick, Ruffing, Seurin. MuSig2: Simple Two-Round Schnorr Multi-Signatures. 2020.
7. Poelstra, A. Scriptless Scripts. 2017. Original adaptor-signature construction.
8. Wuille, P., Nick, J., Towns, A. BIP-340 / 341 / 342: Schnorr signatures and Taproot for Bitcoin.
9. Fournier, L. One-Time Verifiably Encrypted Signatures (ECDSA adaptor signatures). 2019.
10. Nostr. NIP-01, NIP-04, NIP-13, NIP-33, NIP-40. github.com/nostr-protocol/nips
11. Hoytech. strfry. High-performance Nostr relay. github.com/hoytech/strfry

This document is a living specification. Discrepancies between this document and the reference implementation in [pricoinfooundation/pricoins](https://pricoinfooundation.com/pricoins) should be resolved in favor of the implementation, with this document treated as the documentation that needs updating.